

Generador de Exámenes- Backend API (Arquitectura de Microservicios)

Esta es la plataforma completa (Backend y Frontend) para el proyecto "Generador de Exámenes". El sistema está diseñado bajo una arquitectura orientada a microservicios en el lado del servidor, proporcionando un entorno seguro y escalable, acompañado de un cliente web ligero y eficiente.

Arquitectura del Sistema

El backend ha evolucionado de un monolito a una arquitectura de microservicios utilizando el ecosistema Spring Cloud:

1. **Eureka Server (Puerto 8761):** Servidor de descubrimiento de servicios. Mantiene el registro de todas las instancias activas.
2. **API Gateway (Puerto 8080):** Punto de entrada único para el Frontend. Enruta las peticiones a los microservicios correspondientes y maneja las políticas de CORS.
3. **Microservicio de Usuarios (Puerto 8081):** Gestiona la autenticación, autorización (JWT), operaciones CRUD de usuarios y la asignación de roles (RBAC).
4. **Microservicio de Incidencias (Puerto 8082):** Servicio dedicado a la auditoría. Recibe y persiste automáticamente los errores lanzados por el resto de microservicios.
5. **Frontend:** Cliente web desarrollado nativamente sin frameworks pesados, consumiendo los endpoints a través del API Gateway.

Características destacadas

Seguridad integral con JWT (Spring Security): Implementación de un sistema de autenticación sin estado (stateless). Las rutas están fuertemente protegidas mediante Control de Acceso Basado en Roles (RBAC) y validaciones de negocio estrictas.

Subsistema Automático de Incidencias: Un interceptor global (`@RestControllerAdvice`) captura cualquier excepción del sistema. Mediante el uso de `RestTemplate`, el microservicio afectado envía el contexto del error al microservicio de Incidencias de forma asíncrona.

Exportación de Datos: Generación dinámica de reportes en formato Excel (`.xlsx`) utilizando Apache POI para la descarga de registros de usuarios, alumnos y rankings.

Paginación y Ordenación Dinámica: Implementación de `Pageable` y `Sort` en los endpoints de listado para optimizar el rendimiento y permitir la ordenación ascendente o descendente por múltiples criterios.

Logs Centralizados no intrusivos: Uso de Programación Orientada a Aspectos (Spring AOP) para interceptar las llamadas a los servicios y registrar los tiempos de ejecución mediante SLF4J en un archivo `.txt` local.

Alta cobertura de tests: Pruebas unitarias avanzadas con JUnit y Mockito, mockeando incluso el contexto de seguridad (`Authentication`) para validar las reglas de negocio.

Endpoints de la API

Autenticación (/auth) - *Públicos*

Método	Ruta	Descripción
--------	------	-------------

POST	/auth/login	Valida credenciales y devuelve un Token JWT de sesión.
POST	/auth/register	Registra un nuevo usuario en el sistema (Rol ALUMNO por defecto).

Usuarios (/usuarios) - *Protegidos*

Método	Ruta	Descripción
--------	------	-------------

GET	/usuarios	Obtiene la lista de todos los usuarios activos.
GET	/usuarios/paginados	Obtiene la lista de usuarios mediante paginación dinámica.
GET	/usuarios/{id}	Busca un usuario concreto por su ID.
GET	/usuarios/email/{email}	Busca un usuario por su correo electrónico.
PUT	/usuarios/{id}	Modifica los datos del usuario (Aplica reglas ABAC según el Token).
DELETE	/usuarios/{id}	Realiza un borrado lógico del usuario.

| GET| `/usuarios/exportar/excel` | Descarga el listado de usuarios en formato .xlsx. |

Roles(/roles) - *Solo Administradores*

Método	Ruta	Descripción
--------	------	-------------

---	---	---
-----	-----	-----

GET	/roles	Obtiene todos los roles activos del sistema.
-----	--------	--

GET	/roles/{id}	Busca un rol por su ID.
-----	-------------	-------------------------

POST	/roles	Crea un nuevo rol.
------	--------	--------------------

PUT	/roles/{id}	Modifica el nombre o estado de un rol.
-----	-------------	--

DELETE	/roles/{id}	Realiza un borrado lógico del rol.
--------	-------------	------------------------------------

RelacionesUsuario-Rol - *Solo Administradores*

Método	Ruta	Descripción
--------	------	-------------

---	-----	---
-----	-------	-----

GET	/usuarios_rols	Consulta todos los roles que tiene asignados un usuario.
-----	----------------	--

Incidencias(/incidencias) - *Solo Administradores*

Método	Ruta	Descripción
--------	------	-------------

---	---	---
-----	-----	-----

GET	/incidencias	Obtiene el historial completo de errores de la API.
-----	--------------	---

GET	/incidencias/{id}	Busca una incidencia por ID.
-----	-------------------	------------------------------

GET	/incidencias/usuario/{idUserio}	Historial de errores provocados por un usuario específico.
-----	---------------------------------	--

GET	/incidencias/clase/{clase}	Filtra incidencias originadas en una clase concreta.
-----	----------------------------	--

GET	/incidencias/metodo/{metodo}	Filtra incidencias originadas en un método concreto.
-----	------------------------------	--

Stack Tecnológico

- * Java21+

- * Spring Boot 4.0.2

- * Spring Cloud (Netflix Eureka, API Gateway)

- * Spring Security + JWT (io.jsonwebtoken)

- * Spring AOP (AspectJ)

- * Spring Data JPA/ Hibernate

- * MapStruct

- * Apache POI

- * JUnit 5 y Mockito

- * **Frontend:** HTML5, CSS3 (con Tailwind), JavaScript (Vanilla JS)

- * **Base de Datos:** MySQL 8.0

Cómo ejecutar el proyecto en local

1. Requisitos Previos

- * Java21 instalado en el sistema.

- * Servidor MySQL ejecutándose en el puerto 3306.

- * Configurar la variable de entorno `SECRET\_KEY` en el sistema operativo o en el IDE con una cadena segura para la firma de los tokens JWT.

2. Base de Datos

Las bases de datos se generarán automáticamente al iniciar los microservicios gracias a la configuración de Hibernate. El microservicio de usuarios inyectará automáticamente los roles por defecto y un usuario administrador mediante el archivo `data.sql` incluido en el classpath.

3. Ejecución del Backend

El orden de inicio de los servicios es estricto para garantizar el correcto registro y enrutamiento:

1. Ejecutar **EurekaServer** (`localhost:8761`).
2. Ejecutar **API Gateway** (`localhost:8080`).
3. Ejecutar los microservicios de negocio: **Usuarios Service** (`localhost:8081`) e **Incidencias Service** (`localhost:8082`).

La documentación interactiva de la API estará disponible en `http://localhost:8080/swagger-ui.html`.

4. Ejecución del Frontend

Al ser una aplicación desarrollada en JavaScript Vanilla, no requiere instalación de dependencias mediante gestores de paquetes.

1. Navegar al directorio raíz del frontend.
2. Abrir el archivo `index.html` principal directamente en un navegador web moderno, o preferiblemente, servir la carpeta utilizando un servidor local ligero (como **Live Server** en VSCode) para un correcto manejo de las peticiones asíncronas hacia el API Gateway.